

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG



Hội thi Toán học – Lý thuyết và Ứng dụng năm học 2025–2026

Chủ đề: “Toán học trong kỷ nguyên trí tuệ nhân tạo”

CODE NGUỒN DỮ LIỆU
TRÍCH ĐOẠN MÃ PYTHON TÀNG TRUY XUẤT
DỮ LIỆU HOSE – ỨNG DỤNG FINSCOPE

ĐƠN VỊ: KHOA TOÁN – THỐNG KÊ

NHÓM SINH VIÊN THỰC HIỆN:

1. Nguyễn Thành Danh (Trưởng nhóm) — C2300014
2. Trần Huỳnh Nhã Trúc — C2300189

TP. Hồ Chí Minh, tháng 5 năm 2026

1. Giới thiệu

FinScope sử dụng dữ liệu thời gian thực 53 mã chứng khoán niêm yết trên Sở Giao dịch Chứng khoán Thành phố Hồ Chí Minh (HOSE) thông qua thư viện mã nguồn mở `vnstock 4.0.4`. Toàn bộ logic **lấy – làm sạch – chuẩn hoá** dữ liệu được tập trung tại thư mục `data/` của mã nguồn. Tài liệu này trích đoạn ba file cốt lõi của tầng dữ liệu để bạn giám khảo đối chiếu:

- `data/_clients.py` – singleton kết nối + bộ điều tiết tốc độ truy vấn (token bucket) dưới 20 yêu cầu/phút.
- `data/fetcher.py` – truy xuất giá lịch sử 1 mã, kèm các chỉ báo kỹ thuật cơ bản (MA, RSI).
- `data/market.py` – truy xuất snapshot thị trường (giá khớp lệnh hiện tại, vốn hoá, room nước ngoài) cho danh sách mã.

Toàn bộ phần còn lại của ứng dụng (14 trang giao diện, 8 mô hình dự báo, 4 thuật toán toán – tài chính cổ điển, hệ thống xác thực và cá nhân hoá) chạy trực tiếp trên **Streamlit Community Cloud** – bạn giám khảo truy cập qua trình duyệt mà không cần cài đặt môi trường Python (chi tiết tại tệp `Mo_ta_san_pham.pdf`).

2. `data/_clients.py` – Singleton client và token bucket

File này giải quyết hai bài toán: (i) tái sử dụng kết nối `vnstock` – khởi tạo một lần, dùng chung mọi nơi (tránh TLS handshake 200–500 ms cho mỗi lần truy vấn); (ii) chia sẻ một *token bucket* duy nhất giữa mọi luồng (foreground + warmer nền) để tổng tốc độ không vượt giới hạn của `vnstock`.

```

1  """Singleton vnstock clients + token bucket throttle: mọi module trong app
2  chia sẻ chung 1 Vnstock() root + 1 rate limit toàn cục.
3
4  - `vn_root()` : Vnstock() instance dùng chung (init 1 lần/process).
5  - `vn_stock(symbol)` : stock handle cached theo symbol.
6  - `vn_trading(source)` : trading handle (market snapshot, price board).
7  - `throttle()` / `throttle_fg()` : token bucket ≤ 20 req/min (vnstock
8    ceiling). throttle_fg dùng interval ngắn hơn cho user-initiated fetch.
9  """
10 from __future__ import annotations
11 import threading
12 import time
13
14 import streamlit as st
15
16
17 # — Token bucket throttle – chia sẻ giữa MỌI thread (preload, warmer, foreground)
18 MIN_INTERVAL_SEC = 3.4 # warmer/background: 17.6 req/min < 20 ceiling
19 MIN_INTERVAL_FG_SEC = 1.6 # foreground: up to 37 r/m worst-case; thực
20                             # tế chỉ chậm khi user spam ticker. Tradeoff:
21                             # bỏ freeze 3.4s khi đổi mã lần đầu.
22 _LAST_CALL = [0.0]
23 _LOCK = threading.Lock()
24 # Foreground priority: khi user click ticker mới, set tới deadline → warmer
25 # nhường nhịn (đợi tới đó mới xin token). Bỏ race-condition fetch ngắt nhau.
26 _FG_PRIO_UNTIL = [0.0]
27
28
29 def throttle(min_interval: float = MIN_INTERVAL_SEC,
30             is_foreground: bool = False) -> None:
31     """Block tới khi đảm bảo cách lần API call trước ≥ `min_interval`.
```

```

32
33 Thread-safe. Tất cả vnstock-touching code phải gọi throttle() ngay
34 trước khi gọi `.quote.history()` / `.price_board()` / `.finance.*`.
35
36 Khi ``is_foreground=True``: dùng ``MIN_INTERVAL_FG_SEC`` (ngắn hơn) +
37 đặt nhân ưu tiên – warmer/background gặp nhân này sẽ đợi cho tới khi
38 foreground claim xong slot. UX: bỏ giạt khi user đổi mã.
39
40 AppTest / pytest detect → bypass để không block test suite.
41 """
42 import os
43 if os.environ.get('STREAMLIT_TEST') == '1' or os.environ.get('PYTEST_CURRENT_TEST'):
44     return
45 if is_foreground:
46     min_interval = MIN_INTERVAL_FG_SEC
47     _FG_PRIO_UNTIL[0] = time.time() + 0.8 # foreground giữ ưu tiên 0.8s
48 else:
49     # Background/warmer: nhường foreground nếu đang trong giai đoạn ưu tiên.
50     while time.time() < _FG_PRIO_UNTIL[0]:
51         time.sleep(0.2)
52 with _LOCK:
53     dt = time.time() - _LAST_CALL[0]
54     if dt < min_interval:
55         time.sleep(min_interval - dt)
56     _LAST_CALL[0] = time.time()
57
58
59 def throttle_fg() -> None:
60     """Tiện ích cho code foreground (user-initiated fetch)."""
61     throttle(is_foreground=True)
62
63
64 @st.cache_resource(show_spinner=False)
65 def vn_root():
66     """Vnstock() root – singleton trên toàn process. Init 1 lần, share."""
67     from vnstock import Vnstock
68     return Vnstock()
69
70
71 @st.cache_resource(show_spinner=False, max_entries=128)
72 def vn_stock(symbol: str, source: str = 'VCI'):
73     """Stock handle cho 1 ticker – cached_resource theo (symbol, source).
74     Mỗi ticker chỉ init 1 lần / process, không phải mỗi call.
75     """
76     return vn_root().stock(symbol=symbol, source=source)
77
78
79 @st.cache_resource(show_spinner=False)
80 def vn_trading(source: str = 'VCI'):
81     """Trading handle cho market snapshot – singleton."""
82     from vnstock import Trading
83     return Trading(source=source)

```

3. data/fetcher.py – Truy xuất giá lịch sử và chỉ báo

Hàm `fetch_data(ticker)` là cửa ngõ chính để mọi tầng phía trên (mô hình, biểu đồ, bộ máy tín hiệu) lấy chuỗi giá. Kết quả được cache đĩa 24 giờ thông qua `@st.cache_data(persist="disk")`; các chỉ báo cơ bản (MA5/20/50, MA5_Vol, RSI14) được tính ngay tại tầng dữ liệu để mọi page tái sử dụng.

```

1 import streamlit as st
2 import pandas as pd
3 import numpy as np
4 from core.config import DATA_START, DATA_END, DATA_SOURCE, CACHE_TTL
5
6
7 @st.cache_data(ttl=CACHE_TTL, show_spinner=False, persist="disk")
8 def _fetch_raw(ticker: str) -> pd.DataFrame:
9     """Fetch raw từ vnstock + indicator technical. Cache theo ticker (không
10     phụ thuộc date_from/date_to) → đổi date range KHÔNG gọi lại network.
11
12     Bọc redirect_stdout(StringIO) để NUỐT các thông báo tiếng Việt vnstock
13     spam ra console (quảng cáo Insiders + cảnh báo update). Trên Windows
14     cp1252 các ký tự đó gây UnicodeEncodeError → vờ fetch (đã thấy ở ACB).
15     """
16     import contextlib, io
17     # Singleton stock handle + throttle foreground (1.6s) cho user-initiated
18     # fetch. Try/except bao network call → fail gracefully khi vnstock down /
19     # rate-limit / ticker delisted: trả empty DataFrame đúng schema thay vì
20     # raise – caller check df.empty rồi hiện banner thân thiện.
21     from data._clients import vn_stock, throttle_fg
22     try:
23         with contextlib.redirect_stdout(io.StringIO()),
24             ↪ contextlib.redirect_stderr(io.StringIO()):
25             throttle_fg()
26             s = vn_stock(ticker, DATA_SOURCE)
27             df = s.quote.history(start=DATA_START, end=DATA_END, interval='1D')
28     except Exception:
29         # Trả empty df đúng schema → app phía trên không vỡ.
30         return pd.DataFrame(columns=[
31             'Ngày', 'Close', 'Open', 'High', 'Low', 'Volume',
32             'MA5', 'MA20', 'MA50', 'MA5_Vol',
33             'RSI14', 'MA5_ratio', 'MA20_ratio', 'Volume_ratio',
34             'Range', 'Range_ratio', 'Return',
35             'Close_L1', 'Volume_L1', 'Range_L1', 'Return_L1',
36             'Volume_ratio_L1', 'Range_ratio_L1',
37             'MA5_ratio_L1', 'MA20_ratio_L1', 'RSI14_L1',
38         ])
39     if df is None or df.empty:
40         return pd.DataFrame(columns=['Ngày', 'Close', 'Open', 'High', 'Low', 'Volume'])
41     df = df.rename(columns={
42         'time': 'Ngày', 'close': 'Close', 'open': 'Open',
43         'high': 'High', 'low': 'Low', 'volume': 'Volume',
44     })
45     df['Ngày'] = pd.to_datetime(df['Ngày']).dt.date
46     df = df.sort_values('Ngày').reset_index(drop=True)
47
48     # Technical indicators
49     df['MA5'] = df['Close'].rolling(5).mean()
50     df['MA20'] = df['Close'].rolling(20).mean()
51     df['MA50'] = df['Close'].rolling(50).mean()
52     df['MA5_Vol'] = df['Volume'].rolling(5).mean()
53
54     delta = df['Close'].diff()
55     gain = delta.clip(lower=0).rolling(14).mean()
56     loss = (-delta.clip(upper=0)).rolling(14).mean()
57     df['RSI14'] = 100 - 100 / (1 + gain / loss.replace(0, np.nan))
58     # Giá đóng yên 14 phiên (gain==loss==0) → RSI = NaN. dropna() sau đó
59     # sẽ XÓA SẠCH phiên đầu chart cho mã thanh khoản kém → silent data loss.
60     # Fillna 50 (trung tính: không tăng/giảm) → giữ phiên, dùng được L1 lag.

```

```

60 df['RSI14'] = df['RSI14'].fillna(50.0)
61
62 df['MA5_ratio'] = (df['Close'] / df['MA5'] - 1) * 100
63 df['MA20_ratio'] = (df['Close'] / df['MA20'] - 1) * 100
64 df['Volume_ratio'] = df['Volume'] / df['MA5_Vol']
65 df['Range'] = df['High'] - df['Low']
66 df['Range_ratio'] = (df['High'] - df['Low']) / df['Close'] * 100
67 df['Return'] = df['Close'].pct_change() * 100
68
69 # Lag-1 features (for MLR and CART)
70 df['Close_L1'] = df['Close'].shift(1)
71 df['Volume_L1'] = df['Volume'].shift(1)
72 df['Range_L1'] = df['Range'].shift(1)
73 df['Return_L1'] = df['Return'].shift(1)
74 df['Volume_ratio_L1'] = df['Volume_ratio'].shift(1)
75 df['Range_ratio_L1'] = df['Range_ratio'].shift(1)
76 df['MA5_ratio_L1'] = df['MA5_ratio'].shift(1)
77 df['MA20_ratio_L1'] = df['MA20_ratio'].shift(1)
78 df['RSI14_L1'] = df['RSI14'].shift(1)
79
80 df = df.dropna().reset_index(drop=True)
81
82 # Guard: nếu phiên cuối là HÔM NAY và HOSE chưa đóng cửa (trước 15:00 VN),
83 # loại bỏ hàng cuối vì Close là giá intraday chưa chốt chính thức.
84 import datetime as _dt
85 try:
86     from zoneinfo import ZoneInfo
87     _now_vn = _dt.datetime.now(ZoneInfo('Asia/Ho_Chi_Minh'))
88 except Exception:
89     _now_vn = _dt.datetime.utcnow() + _dt.timedelta(hours=7)
90
91 if len(df) > 0:
92     _last_row_date = df['Ngày'].iloc[-1]
93     _today_vn = _now_vn.date()
94     # HOSE đóng 14:45; buffer tới 15:00 để chắc chắn API đã cập nhật giá chốt
95     _close_cutoff = _dt.time(15, 0)
96     if _last_row_date == _today_vn and _now_vn.time() < _close_cutoff:
97         df = df.iloc[:-1].reset_index(drop=True)
98
99 return df
100
101
102 def fetch_data(ticker: str, date_from=None, date_to=None) -> pd.DataFrame:
103     """Public fetcher – lấy raw cache + filter theo date range (in-memory)."""
104     df = _fetch_raw(ticker)
105     if date_from:
106         df = df[df['Ngày'] >= date_from]
107     if date_to:
108         df = df[df['Ngày'] <= date_to]
109     return df.reset_index(drop=True)

```

4. data/market.py – Snapshot toàn thị trường

Hàm `market_snapshot(tickers)` dùng cho các trang Tổng quan thị trường, Phân tích cơ bản và Dashboard. Một lần truy vấn `price_board` của `vnstock` trả về dữ liệu cho cả danh sách 53 mã, tiết kiệm thời gian so với truy vấn từng mã một.

```
1 """Tổng quan Thị trường – snapshot 53 mã HOSE: giá, % thay đổi, vốn hóa.
```

```

2
3 Nguồn chính: vnstock Trading.price_board (1 lệnh gọi cho cả 53 mã). Có
4 5-level fallback (VCI → MSN → TCBS → per-ticker history → static JSON
5 commit trong repo) để đảm bảo luôn có dữ liệu kể cả khi mọi API live fail.
6 """
7 import streamlit as st
8 import pandas as pd
9 import numpy as np
10
11 from core.constants import TICKERS, TICKER_INFO
12
13
14 def _sector_of(tk: str) -> str:
15     """Ngành rút gọn (bỏ '. HOSE')."""
16     s = TICKER_INFO.get(tk, '')
17     return s.split('.')[0].strip() if s else 'Khác'
18
19
20 _EMPTY_COLS = ['ticker', 'sector', 'ref_price', 'last_price', 'change',
21                'change_pct', 'volume', 'value_M', 'market_cap_B', 'listed_share']
22
23
24 def _snapshot_fallback_from_history(symbols: tuple) -> pd.DataFrame:
25     """Fallback khi MỌI Trading source fail (Streamlit Cloud thường gặp):
26     fetch giá đóng cửa mới nhất qua quote.history cho từng mã (cache disk
27     24h sẵn nên rất nhanh sau lần đầu). Trả schema giống price_board nhưng
28     ref_price = previous close, last_price = latest close, không có
29     listed_share/volume realtime → market_cap_B = 0 (caller hide column).
30     """
31     from data.fetcher import fetch_data
32     rows = []
33     for tk in symbols:
34         try:
35             df = fetch_data(tk)
36             if df is None or df.empty or len(df) < 2:
37                 continue
38             last = float(df['Close'].iloc[-1]) * 1000 # nghìn đ → đồng
39             ref = float(df['Close'].iloc[-2]) * 1000
40             vol = float(df['Volume'].iloc[-1])
41             chg = last - ref
42             pct = (chg / ref * 100) if ref > 0 else 0.0
43             rows.append({
44                 'ticker': tk, 'sector': _sector_of(tk),
45                 'ref_price': ref, 'last_price': last, 'change': chg,
46                 'change_pct': pct, 'volume': vol,
47                 'value_M': last * vol / 1e6, # đồng → triệu đ
48                 'market_cap_B': 0.0, # cần listed_share – bỏ
49                 'listed_share': 0.0,
50             })
51         except Exception:
52             continue
53     return pd.DataFrame(rows)
54
55
56 def _snapshot_from_static_json(symbols: tuple) -> pd.DataFrame:
57     """Level 5 fallback: đọc snapshot từ data/static/snapshot.json commit
58     trong repo. Instant load, hiển thị data tại thời điểm commit. Generate
59     bằng cách chạy market_snapshot() local rồi commit file lên GitHub.
60     """
61     import json
62     from pathlib import Path

```

```

63 p = Path(__file__).resolve().parent.parent / 'data' / 'static' / 'snapshot.json'
64 if not p.exists():
65     return pd.DataFrame(columns=_EMPTY_COLS)
66 try:
67     with p.open('r', encoding='utf-8') as f:
68         d = json.load(f)
69         rows = [r for r in d.get('tickers', []) if r.get('ticker') in symbols]
70         df = pd.DataFrame(rows)
71         # Đảm bảo có đủ cột schema để caller không vỡ
72         for col in _EMPTY_COLS:
73             if col not in df.columns:
74                 df[col] = 0.0
75         return df[_EMPTY_COLS]
76 except Exception:
77     return pd.DataFrame(columns=_EMPTY_COLS)
78
79
80 def _try_source_with_timeout(source: str, symbols: tuple, timeout_s: float = 3.0):
81     """Gọi vn_trading(source).price_board với HARD TIMEOUT (tránh stuck
82     30s khi network slow). Trả None nếu timeout/fail.
83     """
84     import contextlib, io
85     from concurrent.futures import ThreadPoolExecutor, TimeoutError
86     from data._clients import vn_trading, throttle
87     def _call():
88         with contextlib.redirect_stdout(io.StringIO()),
89             ↪ contextlib.redirect_stderr(io.StringIO()):
90             throttle()
91             return vn_trading(source).price_board(symbols_list=list(symbols))
92     ex = ThreadPoolExecutor(max_workers=1)
93     try:
94         return ex.submit(_call).result(timeout=timeout_s)
95     except (TimeoutError, Exception):
96         return None
97     finally:
98         ex.shutdown(wait=False)
99
100 @st.cache_data(ttl=300, show_spinner=False)
101 def market_snapshot(symbols: tuple) -> pd.DataFrame:
102     """Snapshot 53 mã HOSE → DataFrame schema:
103     ['ticker', 'sector', 'ref_price', 'last_price', 'change', 'change_pct',
104     'volume', 'value_M', 'market_cap_B', 'listed_share'].
105
106     Giá đơn vị đồng (raw từ price_board); value_M triệu đ; market_cap_B tỷ đ.
107
108     Fallback chain: VCI → MSN → TCBS (timeout 3s mỗi nguồn) → static JSON
109     snapshot commit trong repo. Worst case (mọi API live fail): instant
110     load từ JSON, banner offline.
111     """
112     pb = None
113     for source in ['VCI', 'MSN', 'TCBS']:
114         pb = _try_source_with_timeout(source, symbols, timeout_s=3.0)
115         if pb is not None and len(pb) > 0:
116             break
117     # Khi cả 3 live source fail: ưu tiên static JSON (instant) thay vì
118     # per-ticker fetch (53 × throttle = 85s, BGK đợi mãi).
119     if pb is None or len(pb) == 0:
120         return _snapshot_from_static_json(symbols)
121     rows = []
122     for _, r in pb.iterrows():

```

```

123     try:
124         tk = str(r[('listing', 'symbol')])
125         ref = float(r[('listing', 'ref_price')] or 0)
126         shares = float(r[('listing', 'listed_share')] or 0)
127         match = float(r[('match', 'match_price')] or 0) or ref
128         vol = float(r[('match', 'accumulated_volume')] or 0)
129         val = float(r[('match', 'accumulated_value')] or 0)
130         chg = match - ref
131         pct = (chg / ref * 100) if ref > 0 else 0.0
132         mcap_billion = match * shares / 1e9 # đồng → tỷ đồng
133         rows.append({
134             'ticker': tk,
135             'sector': _sector_of(tk),
136             'ref_price': ref,
137             'last_price': match,
138             'change': chg,
139             'change_pct': pct,
140             'volume': vol,
141             'value_M': val / 1e3, # input đã ×1000 → MM đồng
142             'market_cap_B': mcap_billion,
143             'listed_share': shares,
144         })
145     except Exception:
146         continue
147 df = pd.DataFrame(rows)
148 return df
149
150
151 def market_kpis(df: pd.DataFrame) -> dict:
152     """KPI tổng hợp toàn thị trường mẫu."""
153     if df.empty:
154         return {'n': 0, 'n_up': 0, 'n_down': 0, 'n_flat': 0,
155                 'avg_pct': 0.0, 'total_mcap_T': 0.0, 'total_value_B': 0.0}
156     n_up = int((df['change_pct'] > 0.05).sum())
157     n_down = int((df['change_pct'] < -0.05).sum())
158     n_flat = len(df) - n_up - n_down
159     return {
160         'n': len(df),
161         'n_up': n_up,
162         'n_down': n_down,
163         'n_flat': n_flat,
164         'avg_pct': float(df['change_pct'].mean()),
165         'total_mcap_T': float(df['market_cap_B'].sum() / 1000), # tỷ → nghìn tỷ
166         'total_value_B': float(df['value_M'].sum() / 1000), # MM → tỷ
167     }
168
169
170 def sector_overview(df: pd.DataFrame) -> pd.DataFrame:
171     """Gộp theo ngành: avg %, tổng market cap, số mã, advancers/decliners."""
172     if df.empty:
173         return pd.DataFrame()
174     g = df.groupby('sector', dropna=False)
175     out = g.agg(
176         n=('ticker', 'count'),
177         avg_pct=('change_pct', 'mean'),
178         mcap_B=('market_cap_B', 'sum'),
179         value_M=('value_M', 'sum'),
180         n_up=('change_pct', lambda s: int((s > 0.05).sum())),
181         n_down=('change_pct', lambda s: int((s < -0.05).sum())),
182     ).reset_index()
183     return out.sort_values('mcap_B', ascending=False).reset_index(drop=True)

```



```
184
185
186 def top_movers(df: pd.DataFrame, n: int = 5) -> tuple:
187     """(gainers_df, losers_df) – top n theo change_pct."""
188     if df.empty:
189         return pd.DataFrame(), pd.DataFrame()
190     return (df.nlargest(n, 'change_pct').reset_index(drop=True),
191             df.nsmallest(n, 'change_pct').reset_index(drop=True))
```

5. Ghi chú thực thi

Hai chi tiết kỹ thuật đáng chú ý:

1. **Bao contextlib.redirect_stdout/redirect_stderr:** vnstock in các thông báo bằng tiếng Việt ra console (cảnh báo cập nhật, quảng cáo bản Insiders). Trên hệ điều hành Windows với mã hoá cp1252, các ký tự tiếng Việt gây UnicodeEncodeError làm vỡ hàm fetch. Nhóm bọc các lời gọi vnstock trong khối nuốt stdout/stderr để tránh sự cố này (đã quan sát trên mã ACB).
2. **Tier-aware throttle:** hàm throttle() phân biệt hai loại lời gọi – foreground (is_foreground=True) chỉ chờ tối thiểu 1,6 giây; background/warmer chờ 3,4 giây và nhường nhịn khi foreground vừa claim slot. Cơ chế này giúp khi người dùng đổi mã chứng khoán, ứng dụng không bị treo chờ warmer đã chiếm token trước đó.